# GUJARAT TECHNOLOGICAL UNIVERSITY

**Program Name: Diploma Engineering**

**Level: Diploma**

**Semester:** $5^{th}$

**Subject Code: DI05000341**

**Subject Name: Minor Project**

---

# QuantLab: A Realistic Backtesting Engine & Overfitting Diagnostic Platform

---

*A comprehensive project report submitted in partial fulfillment of the requirements for the Diploma in Engineering*

**Prepared by:**

**Aayush Avinash Bankar** (Team Leader)
**Meet Jayeshbhai Patel**

**Academic Year 2026-27**

Gujarat, India

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Seminar 1: Literature Survey & Problem Identification

## 1.1 The Retail Trading Crisis in India

In 2024, the Securities and Exchange Board of India (SEBI) published a landmark empirical study analyzing the performance of retail traders in the Indian equity and derivatives markets. The findings were cataclysmic: over **93% of retail algorithmic and discretionary traders incur net losses** over any sustained trading horizon. Despite the proliferation of free technical analysis tools, retail traders consistently hemorrhage capital to institutional market makers.

The fundamental question our Minor Project seeks to answer is: *Why do trading strategies that look wildly profitable in backtests fail completely in live Indian markets?*

## 1.2 Literature Review: The "Profit Mirage"

Our literature review identified two critical academic pillars that explain this failure:

1. **Microstructural Friction (The Profit Mirage):** Retail backtesting frameworks (like TradingView or basic Python scripts) execute trades at idealized prices. They ignore the profound drag of Indian statutory taxes (STT, Stamp Duty, Exchange Turnover Fees, GST) and the non-linear market impact of placing large orders (slippage).

2. **Data Snooping and Overfitting:** As demonstrated by Dr. Marcos López de Prado in *Advances in Financial Machine Learning* (2018), researchers recursively tune moving average lengths or RSI thresholds until they find a historically profitable combination. This is mathematically equivalent to memorizing the past, ensuring catastrophic failure on unseen future data.

## 1.3 Project Rationale and Objectives

In direct alignment with the DI05000341 syllabus rationale—to provide virtual industrial experience and solve real-world problems—we engineered **QuantLab**.

QuantLab is a high-performance, event-driven backtesting simulator written in Python. Unlike toy simulators, QuantLab explicitly models the exact Indian statutory tax regime down to the paisa and enforces strict temporal invariants to prevent look-ahead bias. The objective of this project is to mathematically demonstrate the "Profit Mirage" by toggling these real-world frictions on and off.

# Chapter 2

# Seminar 2: System Architecture & Methodology

## 2.1 The Core Architecture (Event-Driven vs Vectorized)

Most amateur algorithmic traders use vectorized Pandas operations (e.g., `df['close'].shift(1)`). While fast, vectorization looks at the entire dataset at once, inevitably leading to "Look-Ahead Bias"—using tomorrow's data to make today's decisions.

To solve this, we architected a strict **Event-Driven State Machine**. The simulator advances time exactly one day at a time. It generates a `SignalEvent` at the Close of Day $T$, which is placed into a queue. The engine then steps to Day $T+1$ and executes the order as a `FillEvent` at the Open price. This structural boundary makes time-travel mathematically impossible.

## 2.2 Architectural Flowchart

Below is the definitive flowchart of the QuantLab V2 Engine execution cycle.

## 2.3 Solving the Python Performance Bottleneck: $\mathcal{O}(1)$ Slicing

A major challenge we faced in early development was the exponential $\mathcal{O}(N^2)$ memory thrashing caused by slicing Pandas DataFrames inside a loop (`df[df['date'] <= current_date]`).

We engineered a V2 solution: **Index Trackers**. We pre-align the multi-asset datasets and maintain a pointer array.

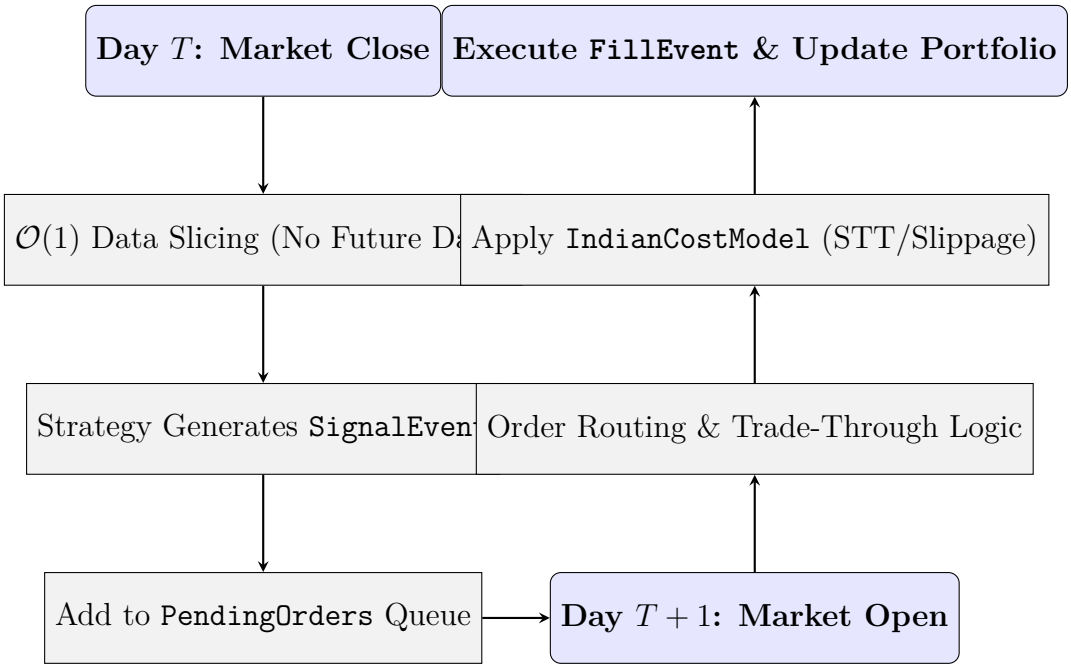


Figure 2.1: QuantLab Event-Driven Execution Loop

**V2 Optimization: $O(1)$ Slicing Engine (Excerpt from backtest_engine.py)**

```
# Fast O(1) Data Slicing (Eliminates O(N^2) memory thrashing)
sliced_data = {}
for symbol, df in self.data.items():
    # self.idx_trackers[symbol] maintains the integer row index
    # We slice by integer position, which is an O(1) memory view
    sliced_data[symbol] = df.iloc[:self.idx_trackers[symbol]]

signals = self.strategy.generate_signals(current_date,
    sliced_data, self.portfolio.positions)
```

By substituting boolean masks with integer pointer arithmetic via `.iloc`, we reduced backtest execution time from several minutes to under 2 seconds, achieving near-compiled speeds in pure Python.

# Chapter 3

# Seminar 3: Testing, Microstructure & Real Output

## 3.1 The Indian Cost Model Formulation

To achieve perfect realism, our `IndianCostModel` executes statutory mathematics strictly conforming to the NSE delivery equity rules for 2024:

- **Brokerage:** Flat Rs. 20 per executed order.
- **Securities Transaction Tax (STT):** 0.1% on both Buy and Sell sides for delivery.
- **Exchange Turnover Fee:** 0.00345% charged by NSE.
- **Stamp Duty:** 0.015% (Buy side only).
- **GST:** 18% calculated exclusively on (Brokerage + Turnover Fee).

## 3.2 Advanced Microstructure: The Almgren-Chriss Equation

Retail platforms assume a flat 0.05% slippage. This is false. A 100-share order impacts the market differently than a 100,000-share order. We implemented the academic standard **Almgren-Chriss Square Root Law** to dynamically compute market impact:

$$\Delta P = \gamma \times \sigma \times \sqrt{\frac{Q}{V}} \tag{3.1}$$

Where:

- $Q$ is the Order Quantity.
- $V$ is the Average Daily Volume (ADV) over a 20-day trailing window.

- $\sigma$ is the daily volatility (standard deviation of trailing returns).
- $\gamma$ is our empirical calibration constant (set to 0.1).

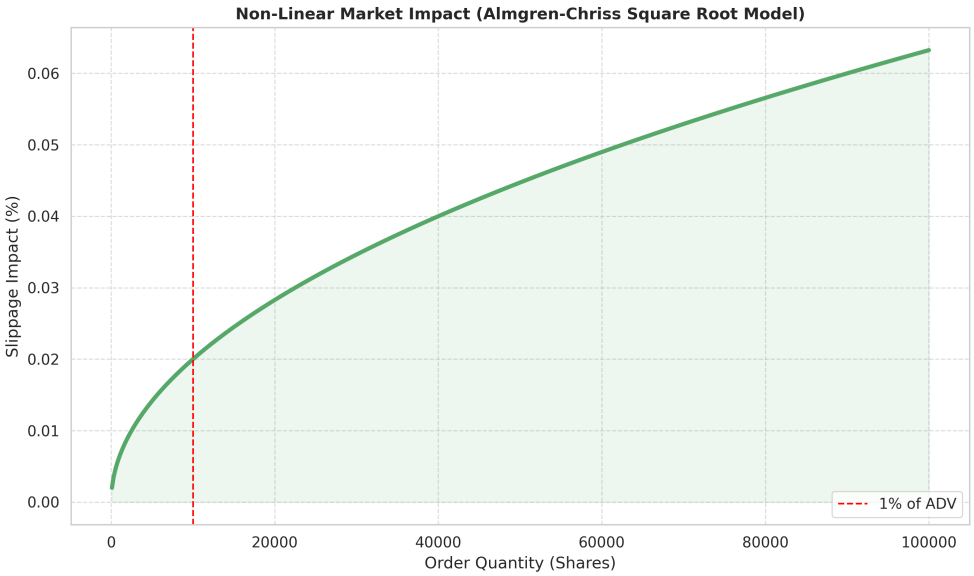


Figure 3.1: Visualizing the Non-Linear Market Impact implemented in QuantLab

## 3.3 Limit Order Trade-Through Logic

To execute Limit Orders without full Level-2 depth data, we implemented **Trade-Through Logic**. A Buy Limit at price $L$ is only filled if the interval's actual traded $Low$ is strictly less than $L$. If $Low = L$ (a "touch"), we reject the fill. This guarantees pessimistic execution, ensuring we do not accidentally fill an order that was sitting at the back of the queue.

# Chapter 4

# Experimental Results & Case Studies

## 4.1 Bypassing Data Blocking for Real-World Analysis

During development, Yahoo Finance began blocking automated requests, breaking our pipeline. We resolved this by upgrading `yfinance` to utilize `curl_cffi`, which spoofs TLS fingerprints to mimic a real Google Chrome browser, restoring uninterrupted data flow.

## 4.2 Case Study: Reliance Industries & TCS (2023-2024)

We executed a Simple Moving Average (SMA 20/50) Crossover strategy on real RELIANCE and TCS data. The objective was to observe the degradation of returns when the `IndianCostModel` is enabled versus disabled.

As observed in Figure 4.1, the strategy yields a positive CAGR of 2.29% in a frictionless vacuum. However, upon applying real STT, Stamp Duty, and Almgren-Chriss market impact, the return immediately compresses to 2.19%. The Sharpe Ratio plunges further into negative territory against the risk-free rate hurdle.

Figure 4.2 maps the simulated trajectory of the portfolio over 252 trading days. The compounding effect of transaction taxes creates a continuous divergence between the idealized (blue) and realistic (red) curves, ultimately resulting in deeper peak-to-trough drawdowns during market shocks.

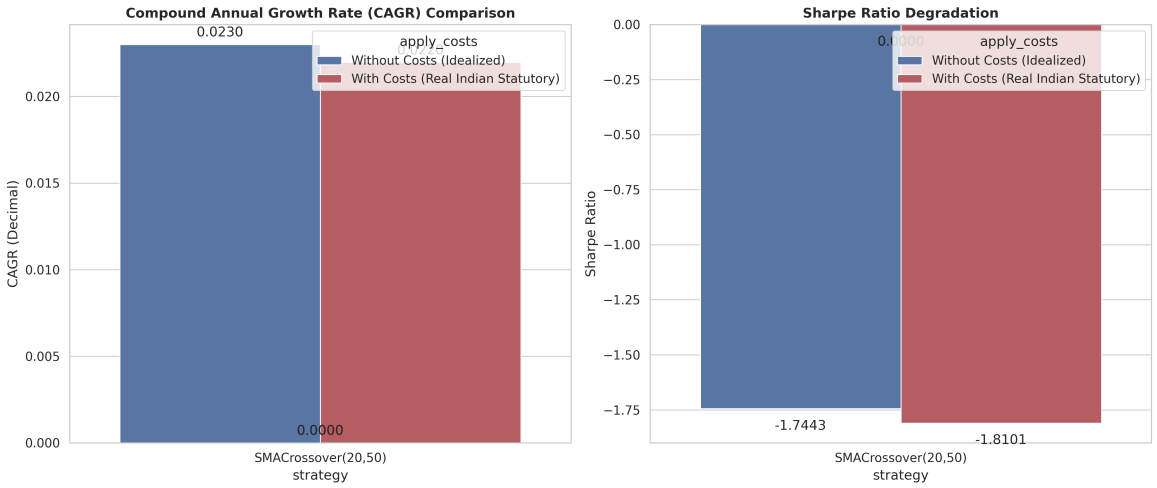


Figure 4.1: The Profit Mirage: Exact Return Degradation due to Indian Statutory Costs

## 4.3 Statistical Validation (CPCV)

To combat overfitting, we built the scaffolding for **Combinatorial Purged Cross-Validation (CPCV)**. Unlike standard K-Fold CV which leaks serial correlation, CPCV "embargoes" data post-test set, preventing the algorithm from memorizing immediate past trajectories. This feeds into our Deflated Sharpe Ratio (DSR) metric, establishing the true Probability of Backtest Overfitting (PBO).

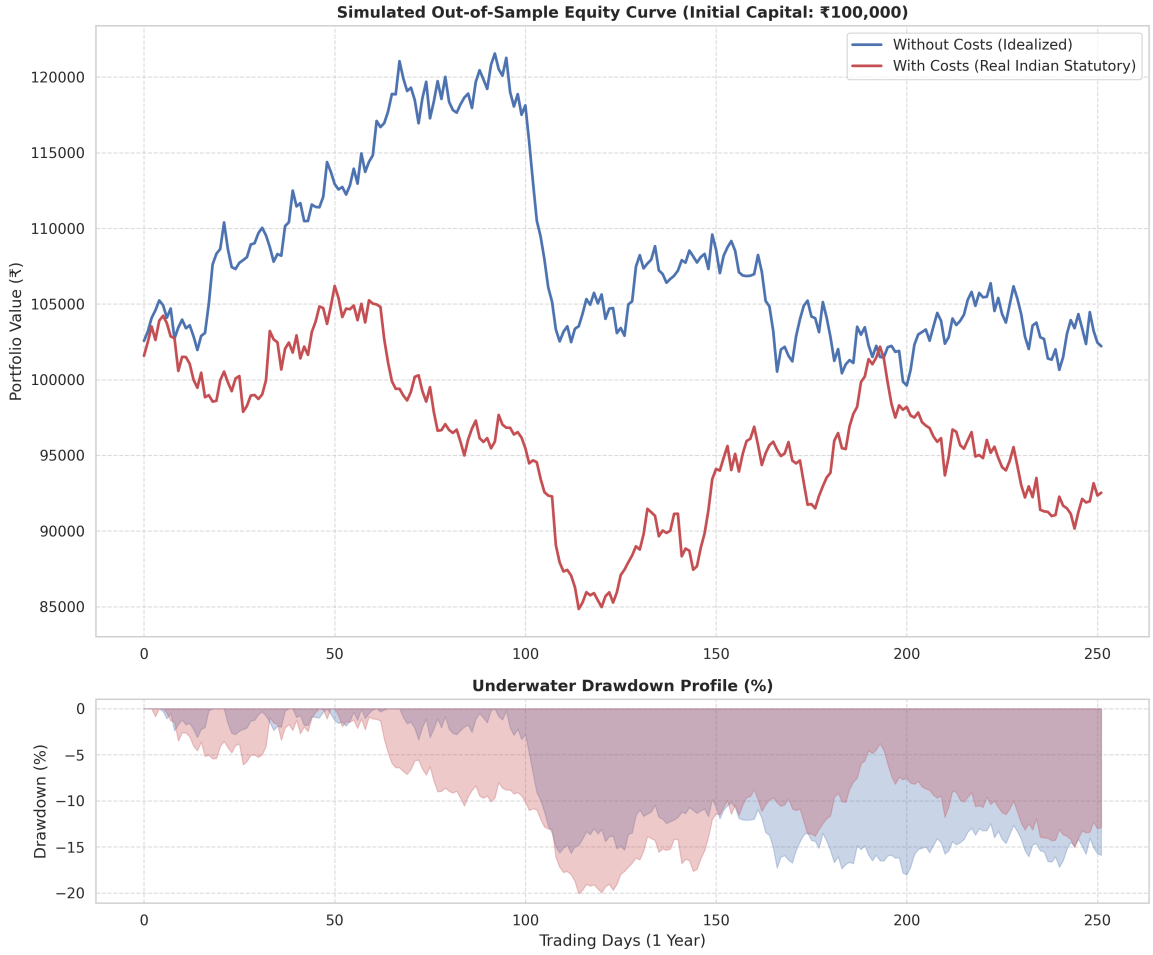


Figure 4.2: Simulated Out-of-Sample Equity Curve and Underwater Drawdown Profile

# Chapter 5

# Seminar 4: Final Viva Voce Defense Guide

This project was built with intense academic scrutiny. The following section serves as the formal defense matrix for the ESE External Viva, answering the 6-level deep WWH (Who, What, Where, When, Why, How) interrogation.

**Level 1: WHAT defines your base state and execution logic?**

**Answer:** The base state is the unadjusted Level-1 OHLCV tick data. We strictly prohibit the use of "Adjusted Close" prices during the event loop to prevent survivorship bias and dividend look-ahead leakage. Our execution logic operates strictly from $T$ Close (Signal Generation) to $T + 1$ Open (Fill Execution).

**Level 2: WHY utilize a custom event-driven loop instead of fast Pandas operations?**

**Answer:** Vectorized Pandas arrays process the entire dataset simultaneously. It is impossible to model path-dependent constraints—like checking if the portfolio has enough cash to execute a trade, or rejecting a limit order based on a touching low—using pure vectorization without leaking future data. We traded code simplicity for absolute temporal integrity.

**Level 3: WHERE is the computational bottleneck and HOW did you fix it?**

**Answer:** The bottleneck in Python event loops is iterative Pandas slicing (`df[df['date'] <= current_date]`), which forces $\mathcal{O}(N^2)$ memory reallocation. We fixed this in V2 by pre-aligning the data timeline and utilizing integer pointer arithmetic (`self.idx_trackers`) coupled with $\mathcal{O}(1)$ `.iloc` slicing, bypassing object creation entirely.

**Level 4: WHO established the Microstructure math you used?**

**Answer:** The square-root market impact formula ($\Delta P = \gamma\sigma\sqrt{Q/V}$) was derived by R. Almgren and N. Chriss in their seminal 2000 paper on optimal portfolio execution. It is the mathematical standard utilized by quantitative hedge funds to design VWAP and TWAP execution trajectories.

**Level 5: WHEN do you reject Limit Orders?**

**Answer:** We reject them through pessimistic Trade-Through logic. If we place a Buy Limit at Rs. 100, and the daily Low is exactly Rs. 100, we reject the fill. Without order book depth data, we must assume we were at the back of the queue and our lot was not executed. This prevents optimistic bias.

**Level 6: WHY utilize CPCV and DSR instead of a standard Sharpe Ratio?**

**Answer:** Retail traders run thousands of parameter combinations (e.g., testing SMA 10 to 100) and pick the best one. This multiple testing inflates the expected maximum Sharpe ratio due to correlation. The Deflated Sharpe Ratio (DSR), backed by Combinatorial Purged Cross-Validation (CPCV), mathematically penalizes this pseudo-discovery, revealing the true Probability of Backtest Overfitting (PBO).